Correlations!

It should be obvious what we are going to do next.

This is "easy". Add correlations to the graphs.

We are going to go back and use linregress to look for correlations between arsenic and other parameters. Then we will plot them.....

This is the hardest stuff we have done. But we are taking two classes. Take your time and try to understand each step. I have all the answers at the end.

In [16]:
```
%matplotlib ipympl
import pandas as pd
import numpy as np
import matplotlib.pylab as plt
from scipy import stats
from matplotlib.backends.backend_pdf import PdfPages
np.set_printoptions(legacy='1.25') #removes the funny printing
```

Get the data in!!!

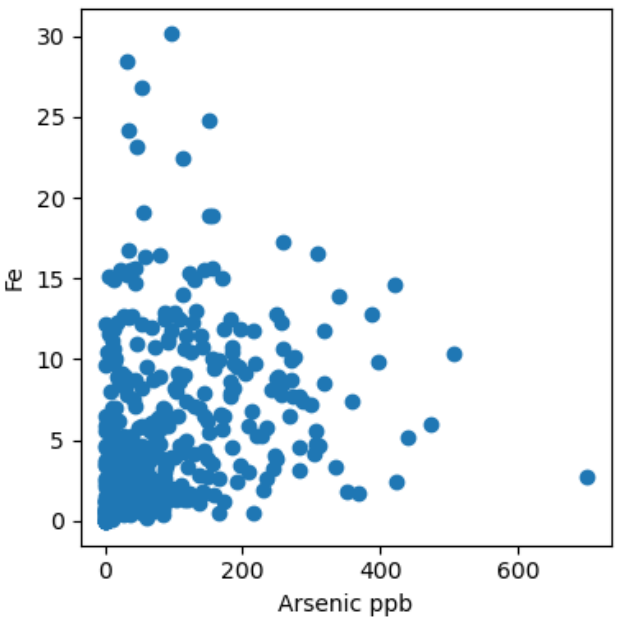
In [2]:
```
df=pd.read_csv('well_data.csv')
```

A few notes. Again, pay attention to the details. Take it slow. Also, today I am using x again for now (I still migh change names later. Don't let that trip you up. You control the names! First lets plot arsenic versus iron

In [3]:
```
fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)  #I made it smaller for printing.  You can skip this line
ax.scatter(df.As,df.Fe)
ax.set_xlabel('Arsenic ppb')
ax.set_ylabel('Fe')
```

Out[3]:
```
Text(0, 0.5, 'Fe')
```

Figure

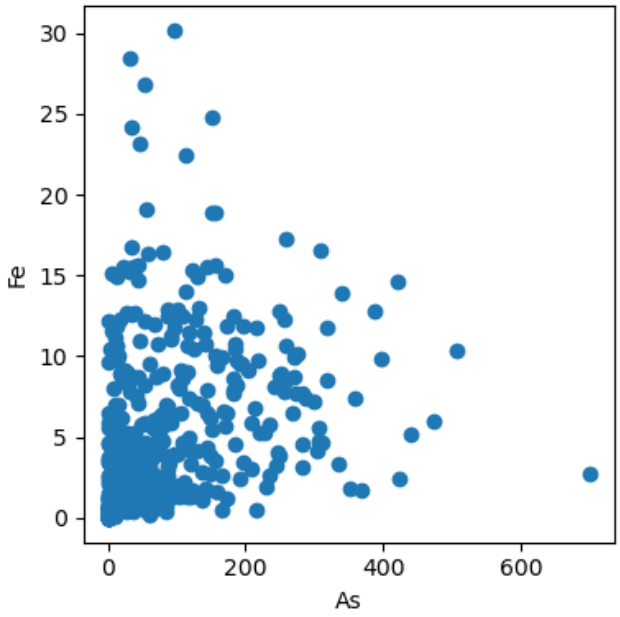


We will want to automate things today. So lets try and use variables for x and y or independent and dependent variables.

In [4]:
```
x='As'
y='Fe'

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)
ax.scatter(df[x],df[y])
ax.set_xlabel(x)
ax.set_ylabel(y)
```

Out[4]:
```
Text(0, 0.5, 'Fe')
```

Figure

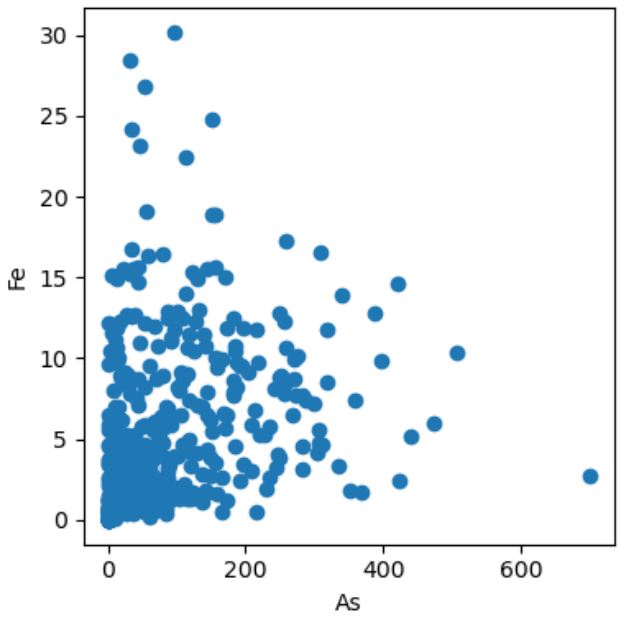


remember you can save figures.

fig.savefig('Name_of_figure.jpg',dpi=200,bbox_inches='tight')

In [5]:
```
x='As'
y='Fe'

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)
ax.scatter(df[x],df[y])
ax.set_xlabel(x)
ax.set_ylabel(y)

fig.savefig('my_first_figure.jpg',dpi=200,bbox_inches='tight')
```

Figure

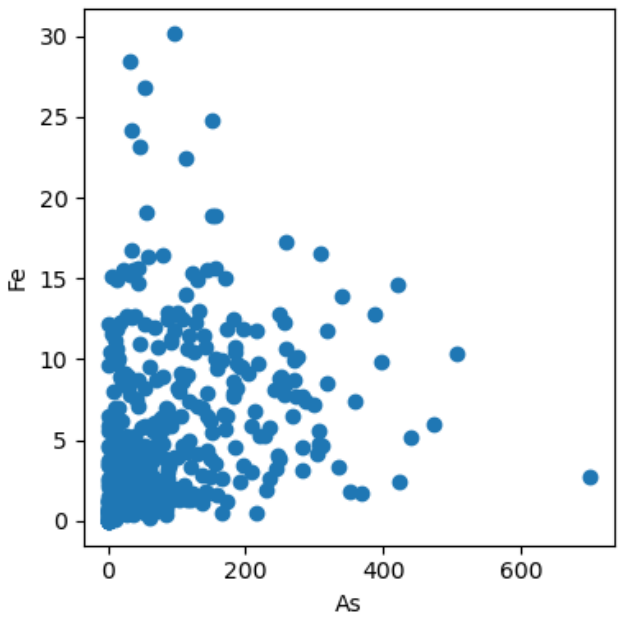


Remember you make up the names. So instead of x and y you could do independent and dependent variable.

In [6]:
```
ind='As'
dep='Fe'

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)
ax.scatter(df[ind],df[dep])
ax.set_xlabel(ind)
ax.set_ylabel(dep)
```

Out[6]:
```
Text(0, 0.5, 'Fe')
```

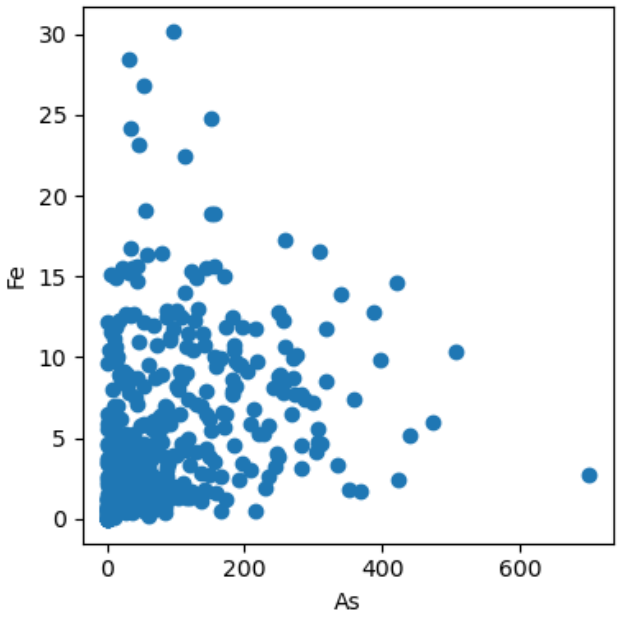
Figure



For later on we will want to make sure we are only plotting continuous data. So before we plot we might want to chack and make sure the y values are a float

In [7]:
```
x='As'
y='Fe'

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

if df[y].dtype==float:
    ax.scatter(df[x],df[y])
    ax.set_xlabel(x)
    ax.set_ylabel(y)
```

Figure



Now lets add linregress to see the correlations. This has some tricks. So we will do it seperately and pull it together. We want stats.linregess(x,y). Unfortunately this gives us NaN or not a number.

In [8]:
```
x='As'
y='Fe'

stats.linregress(df[x],df[y])
```

Out[8]:
```
LinregressResult(slope=np.float64(nan), intercept=np.float64(nan), rvalue=np.float64(nan), pval
ue=np.float64(nan), stderr=np.float64(nan), intercept_stderr=np.float64(nan))
```

These NaN's are becuase stats uses all the data. pandas screens out the NaN's when it does math. So we need to get rid of them. There is a function dropna(). But we need to drop from when either one or both have an NaN. I think it is easier to make a new dataframe for this. So take just As and i and put them into a new pandas dataframe called temp.

In [9]:
```
dfClean=df[[x,y]]
```

In [10]:
```
print (dfClean.head())
```

```
        As        Fe
0      NaN       NaN
1      NaN       NaN
2      NaN       NaN
3  78.97747  1.260031
4      NaN       NaN
```

But we still have nan's. lets make it with Nan dropped. If either have an NaN it removes the row

In [13]:
```
dfClean=df[[x,y]].dropna()
```

In [14]:
```
print (dfClean.head())
```

```
          As         Fe
3   78.977470   1.260031
6   28.070949   1.843156
8   96.885674  11.740445
9   80.627214   8.923465
11  77.006865   6.349396
```

Now try linregress again but on the new temp! Think about how to reference it!

In [17]:
```
x='As'
y='Fe'
dfClean=df[[x,y]].dropna()
stats.linregress(dfClean[x],dfClean[y])
```

Out[17]:
```
LinregressResult(slope=0.01299689964334686, intercept=4.390525620904394, rvalue=0.2560418095014
199, pvalue=1.6310638519922982e-07, stderr=0.0024382468890464413, intercept_stderr=0.3300848122
3672714)
```

This is review for how to make your text box.

Now we are getting close. Look up help to remember what all the output is. Then we can write a caption using fancy print formatting.

- You have done this a few times. But this is a review.
- set results to linregress and you will get a list.
- Then you can make a textstr
- print textstr explaining the results.
- If you type r^2 (This is dollar signs around r^2 to make the math) at some point later on the graphs we will get a real r-squared.

Here is the linregress link to remind you of the output.
http://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.linregress.html

The nice thing with Python 3 is it now gives the names in the output! Oh life is so easy!

remember to get the results you can do results[0] or results.slope to get the same answer or set it equal to m,b,r_value,p_value=....

In [24]:
```

```

```
m=0.013
b=4.391
$r^2$=0.066
p=0.000
```

Now lets print our one graph with the data. For these look at our favorite recipes
http://matplotlib.org/users/recipes.html and scroll all the way to the bottom. This is a reminder of what you need to do.

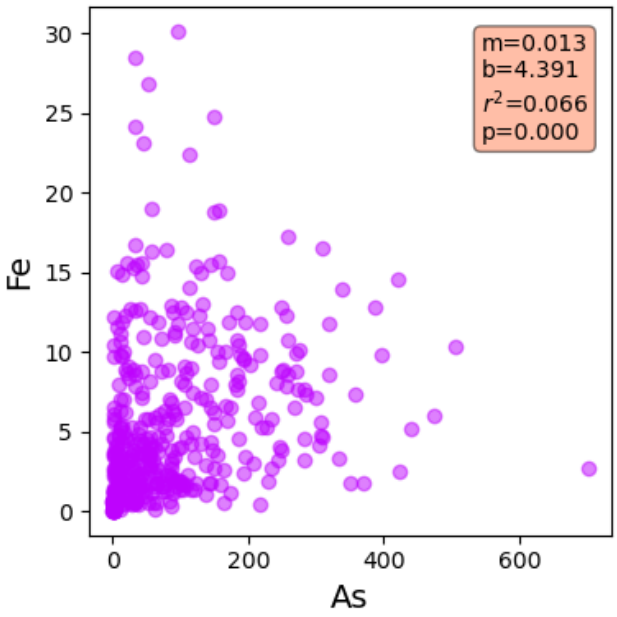
1. Make your text string
2. Make props to give you properties for the box.
3. pass ax.text and place the box.

Use your graph from above and add this stuff in. Plus bring in linregress text string into the same program. You should almost have all the parts!

In [35]:
```

```

Fe

Figure



Now we just need to add the best fit line. You have the m and b from linregress.

Remember we talked about what to use for x_fit. I found the answer to what works best. Rememebr I said you could use

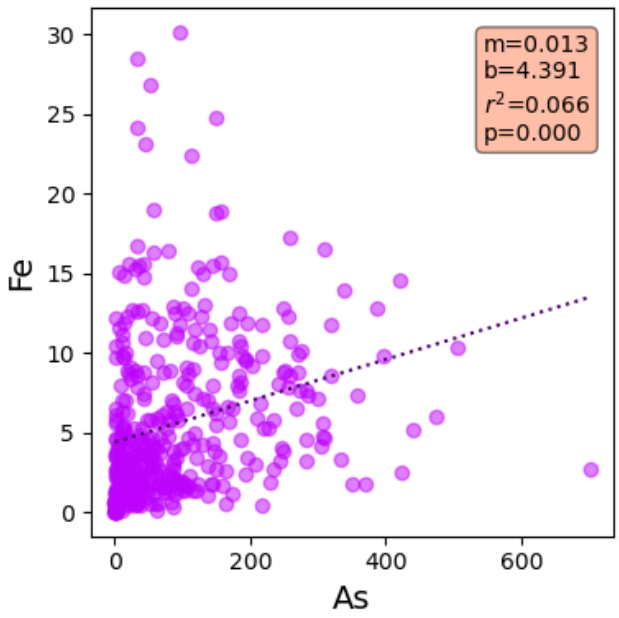
1. your x-values
2. the min and max of the x values
3. Random numbers you make up.

I found out why min and max of the x values works the best. If you use all the x-values it adds a lot of points to the line and then the line looks funny. If you use the min and max of the x-values the line comes out nice. I would do x_fit=np.array([df[x].min() and df[x].max()).

In [38]:

Fe

Figure



Review: The next step is to loop over each column and print the data type. Pandas is crazy. If you put the dataframe in a for loop it goes through the columns automatically.

In [40]:
```
for column in df:
    print(column)
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

I called it

for column in df:

This way I thought col was short for columns. but we could call it anything. literally

In [41]:
```
for anything in df:
    print(anything)
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

But we only want to loop over the elements. So we should start at Si. so can for loop from that column onward. From our notes from last class I know of at lease two ways to choose all columns from Si onward

df.loc[:,'Si':]

df.iloc[:,5:]

In [44]:
```
for column in df.iloc[:,5:]:
    print(column)
```

```
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

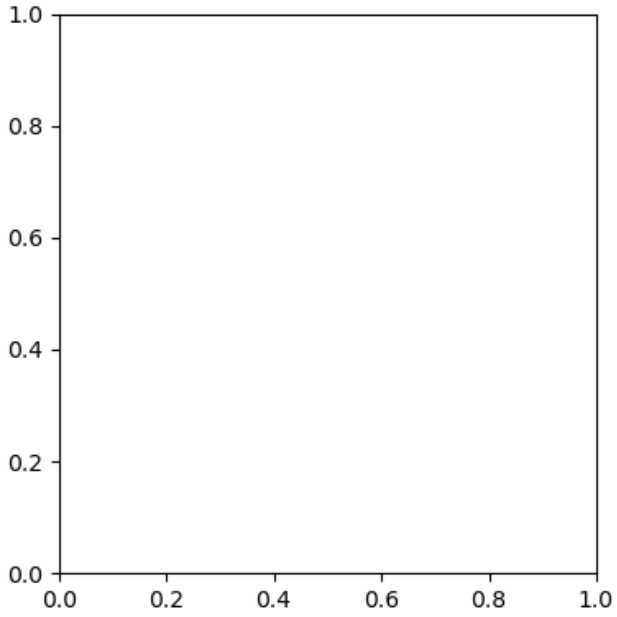
Remember how you did the graphs yesterday? Combine yesterday with today

- one pdf file
- best fit line and data on each graph
- linregress fails when x==y so you need to check for this in the if statement

In [47]:
```

```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



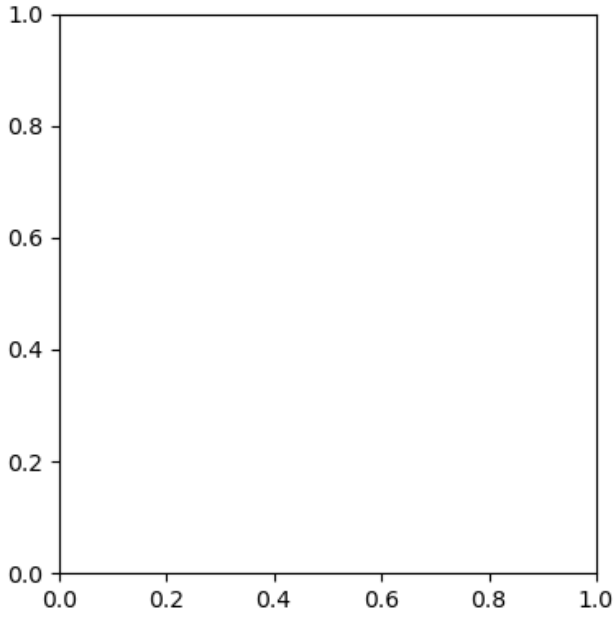
Now add a good title

Add a title that says if significant or not at the 0.01 level

In [52]:

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



Great Job.

I just taught you how to p-hack your data. p-hacking is wrong. I think Taylor Swift said it correctly.

They say I did something bad

Then why's it feel so good?

p-hacking is bad but it is what people do! https://bitesizebio.com/31497/guilty-p-hacking/ So as long as you are concious of what you are doing it is sort of ok. You should really start with a hypothesis and then look for correlations. But it is also alright to explore the data this way. Just be cognizant of the weaknesses.

But this is great. we have all the data. But now we want to take the best 4 or 6 r^2 values and plot them on one sheet. to get the r^2 values we could look through all the graphs. or we can store them and look at them. I don't have a perfect method to do this. I used to make a dictionary. But now I think it is easier to make an empty DataFrame and then fill it in. Here is what I do

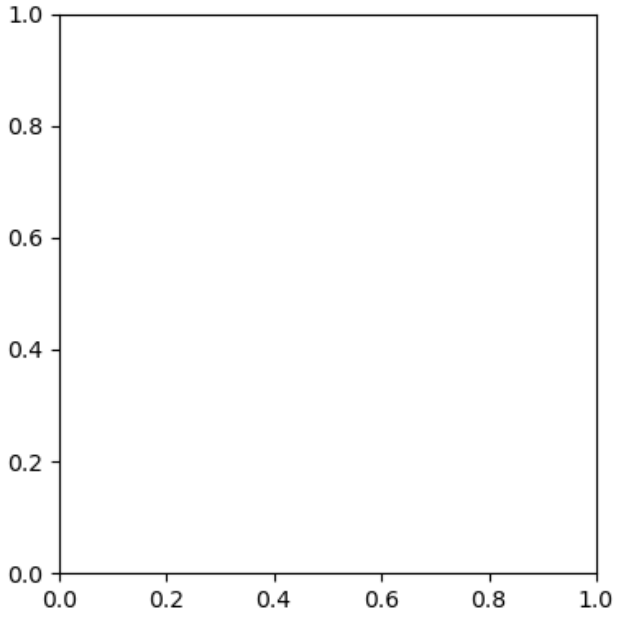
1. Use your last code but add a few lines to save and store the r^2 values.

2. Make a new empty DataFrame with pd.DataFrame() I call mine df_r2 =pd.DataFrame()
3. add to the dataframe each time you loop through.
4. adding to dataframes by rows is sort of a pain.
5. I use .at to add to a dataframe. you do df.at[index,'column']=value
6. You could use a number for the index. But I find it easiest to use the element name since it is unique
7. I made an r2 column and a p-value column because I could.
8. I print out the results in the next cell.

In [18]:
```
pp = PdfPages('regression.pdf')  #open a pdf file

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

df_r2=pd.DataFrame() #empty dataframe

for column in df:

    x='As'
    y=column

    print (y)

    if df[y].dtype==float and x!=y:
        ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
        ax.set_xlabel(x,fontsize=14)
        ax.set_ylabel(y,fontsize=14)

        df_clean=df[[x,y]].dropna()   #drop the not a numbers
        results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

        textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                        format(results.slope,results.intercept\
                               ,results.rvalue**2,results.pvalue)   #make the text string

        ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
                ,verticalalignment='top',bbox=props)     #add the text box

        x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
        ax.plot(x_fit,results.slope*x_fit+results.intercept
                ,color='xkcd:royal purple',linestyle='dotted')  #make the plots

        if results.pvalue<0.01:
            title='{} vs {} is signifanct at the 0.01 level'.format(x,y)
            ax.set_title(title)
        else:
            title='{} vs {} is NOT signifanct at the 0.01 level'.format(x,y)
            ax.set_title(title)

        # Make a new dataframe with r2
        df_r2.at[column,'r2']=results.rvalue**2
        df_r2.at[column,'pvalue']=results.pvalue


        pp.savefig(dpi=200,bbox_inches='tight')
        ax.cla()  #clear the axes to start again

pp.close()  #close the pdf file when done
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



here is the r^2!

In [19]:
```
df_r2
```

Out[19]:

	r2	pvalue
Lat	0.000047	8.904466e-01
Lon	0.002521	3.123120e-01
Si	0.037977	7.576966e-05
P	0.069488	6.760020e-08
S	0.014789	1.409054e-02
Ca	0.154148	1.885646e-16
Fe	0.065557	1.631064e-07
Ba	0.059968	5.689137e-07
Na	0.053342	2.490748e-06
Mg	0.000824	5.636854e-01
K	0.000090	8.609220e-01
Mn	0.032770	2.414060e-04
Sr	0.155595	1.327101e-16
F	0.000030	9.131125e-01
Cl	0.009093	5.925938e-02
SO4	0.021915	3.919941e-03
Br	0.018130	7.993404e-03

Now we need to sort the dataframe.

Here is the link. https://pandas.pydata.org/pandas-docs/stable/generated/pandas.DataFrame.sort_values.html A key thing is the inplace keyword you have not seen before. inplace tells it to sort and keep it. Otherwise it just sorts and shows you and puts it back. You could also set it to itself. So you have two options

- df_r2.sort_values(by='r2',ascending=False,inplace=True)
- df_r2=df_r2.sort_values(by='r2',ascending=False)

In [20]:
```
df_r2.sort_values(by='r2',ascending=False,inplace=True)
```

Once you learn this .at trick it is easy to make a dataframe. I will set the rows to an index and make the column name rsq. Here is a problem.

In [21]:
```
df_r2
```

Out[21]:

	r2	pvalue
Sr	0.155595	1.327101e-16
Ca	0.154148	1.885646e-16
P	0.069488	6.760020e-08
Fe	0.065557	1.631064e-07
Ba	0.059968	5.689137e-07
Na	0.053342	2.490748e-06
Si	0.037977	7.576966e-05
Mn	0.032770	2.414060e-04
SO4	0.021915	3.919941e-03
Br	0.018130	7.993404e-03
S	0.014789	1.409054e-02
Cl	0.009093	5.925938e-02
Lon	0.002521	3.123120e-01
Mg	0.000824	5.636854e-01
K	0.000090	8.609220e-01
Lat	0.000047	8.904466e-01
F	0.000030	9.131125e-01

This is really p-hacking. You almost don't need the graphs. Just the p-values!

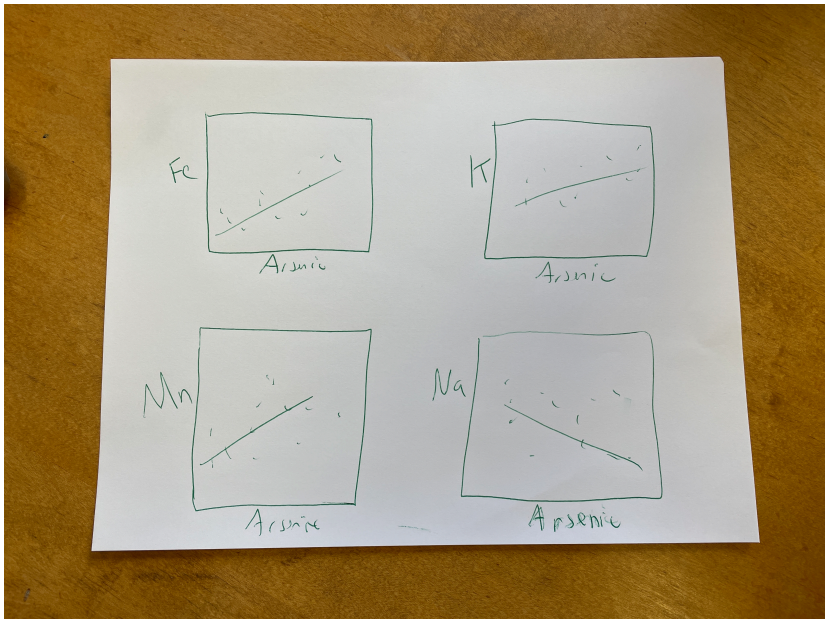
NEW SECTION!

Make lots of graphs next to each other!

This is our Goal

In [28]:
```
from IPython.display import Image
Image(filename='Figure-schematic.JPG',width=500)
```

Out[28]:

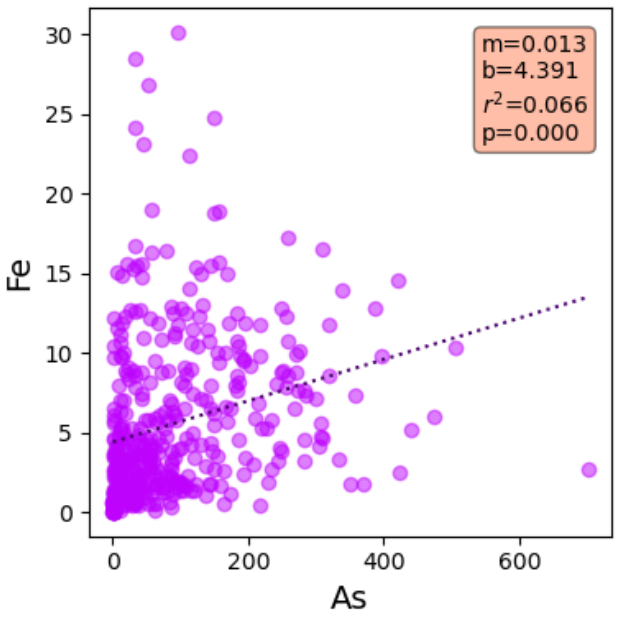


Now lets put our top 4 correlations onto one page! I will walk you through it. Grab your plot from above and strip out the file information and the for loop and the .at. It should just be a plot of x=arsenic versus y. You can set y='Fe'

In [22]:
```
fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

x='As'
y='Fe'

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
ax.set_xlabel(x,fontsize=14)
ax.set_ylabel(y,fontsize=14)

df_clean=df[[x,y]].dropna()   #drop the not a numbers
results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
               format(results.slope,results.intercept\
                     ,results.rvalue**2,results.pvalue)   #make the text string

ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)     #add the text box

x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
ax.plot(x_fit,results.slope*x_fit+results.intercept
        ,color='xkcd:royal purple',linestyle='dotted')  #make the plots
```

Out[22]:
```
[<matplotlib.lines.Line2D at 0x147322250>]
```

Figure



Goal

- We want 4 graphs nicely lined up on one page to print.
- make the subplots 2,2 so we can show four graphs.
- We are going to use a for loop like last time.
- So lets make a list of elemsents we want to graph.
- Grab the top four r^2 elements from above.

First make your subplots(2,2)

You will get an error because ax is a two-d array.

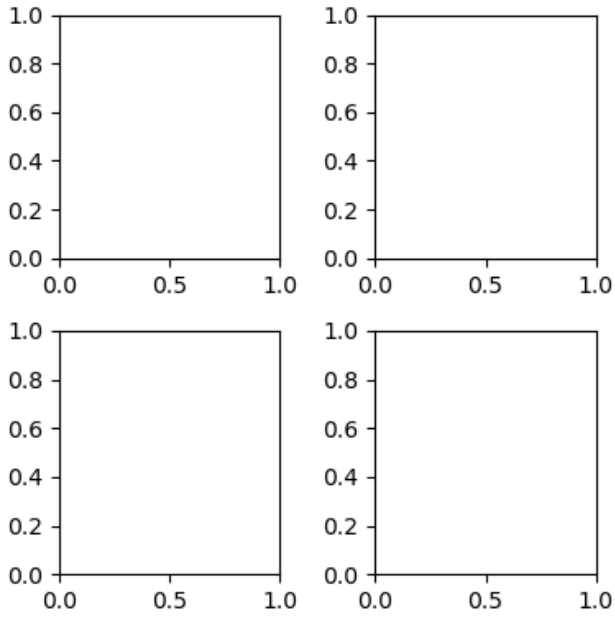
You are going to get an error!!

In [27]:
```
fig,ax=plt.subplots(2,2,layout='tight')
fig.set_size_inches(4,4)

x='As'
y='Fe'

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
ax.set_xlabel(x,fontsize=14)
ax.set_ylabel(y,fontsize=14)

df_clean=df[[x,y]].dropna()   #drop the not a numbers
results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                format(results.slope,results.intercept\
                      ,results.rvalue**2,results.pvalue)   #make the text string

ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)     #add the text box

x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
```

```
ax.plot(x_fit,results.slope*x_fit+results.intercept
        ,color='xkcd:royal purple',linestyle='dotted')  #make the plots
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[27], line 9
      5 y='Fe'
      7 props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box
----> 9 ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
     10 ax.set_xlabel(x,fontsize=14)
     11 ax.set_ylabel(y,fontsize=14)

AttributeError: 'numpy.ndarray' object has no attribute 'scatter'
```

Figure



The problem is now your ax is a 2d array!!!!

Where you call ax after the initial value where you set it make it ax[0,0]

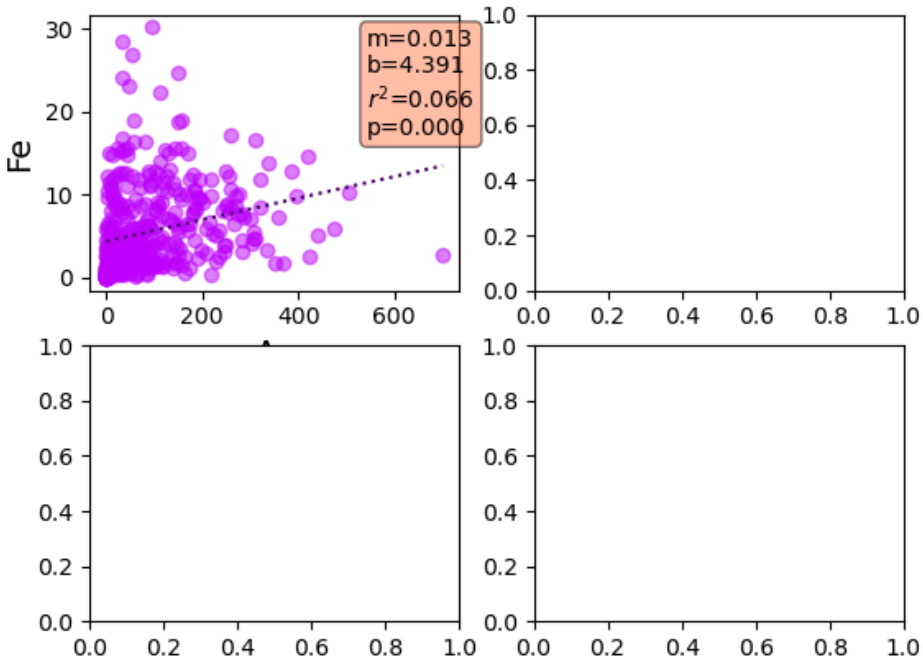
there are a few hidden ax calls!

In [28]:
```
fig,ax=plt.subplots(2,2)

x='As'
y='Fe'

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

ax[0,0].scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
ax[0,0].set_xlabel(x,fontsize=14)
ax[0,0].set_ylabel(y,fontsize=14)

df_clean=df[[x,y]].dropna()   #drop the not a numbers
results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                format(results.slope,results.intercept\
                      ,results.rvalue**2,results.pvalue)   #make the text string

ax[0,0].text(0.75,0.95,textstr,transform=ax[0,0].transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)     #add the text box

x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
```

```
ax[0,0].plot(x_fit,results.slope*x_fit+results.intercept
        ,color='xkcd:royal purple',linestyle='dotted')  #make the plots
```

Out[28]: [<matplotlib.lines.Line2D at 0x148661c10>]

Figure



Next step

That worked nicely. Now go back up and make graph on the next graph over!

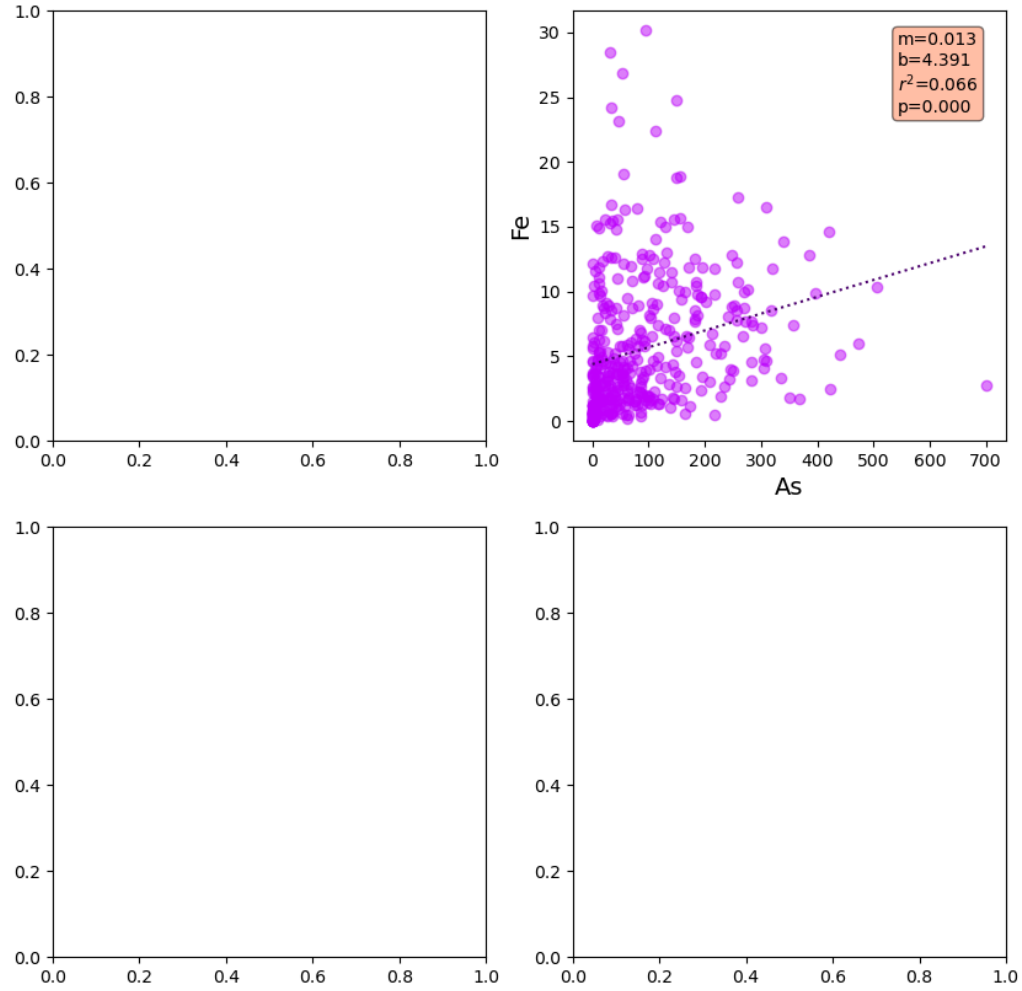
Plus make it bigger using fig.set_size_inches(10,10)

In [29]:
```
fig,ax=plt.subplots(2,2)

fig.set_size_inches(10,10)

x='As'
y='Fe'

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

ax[0,1].scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
ax[0,1].set_xlabel(x,fontsize=14)
ax[0,1].set_ylabel(y,fontsize=14)

df_clean=df[[x,y]].dropna()   #drop the not a numbers
results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                format(results.slope,results.intercept\
                      ,results.rvalue**2,results.pvalue)   #make the text string

ax[0,1].text(0.75,0.95,textstr,transform=ax[0,1].transAxes,fontsize=10\
        ,verticalalignment='top',bbox=props)     #add the text box

x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
```

```
ax[0,1].plot(x_fit,results.slope*x_fit+results.intercept
        ,color='xkcd:royal purple',linestyle='dotted')  #make the plots
```

Out[29]: [<matplotlib.lines.Line2D at 0x148d1aa90>]

Figure



That worked nicely.

But we have a problem. We have our list of 4 elements we want to plot. But we have a 2x2 matrix to put them into when using ax.

confusing!!!!

use the script below to see how the numbering works

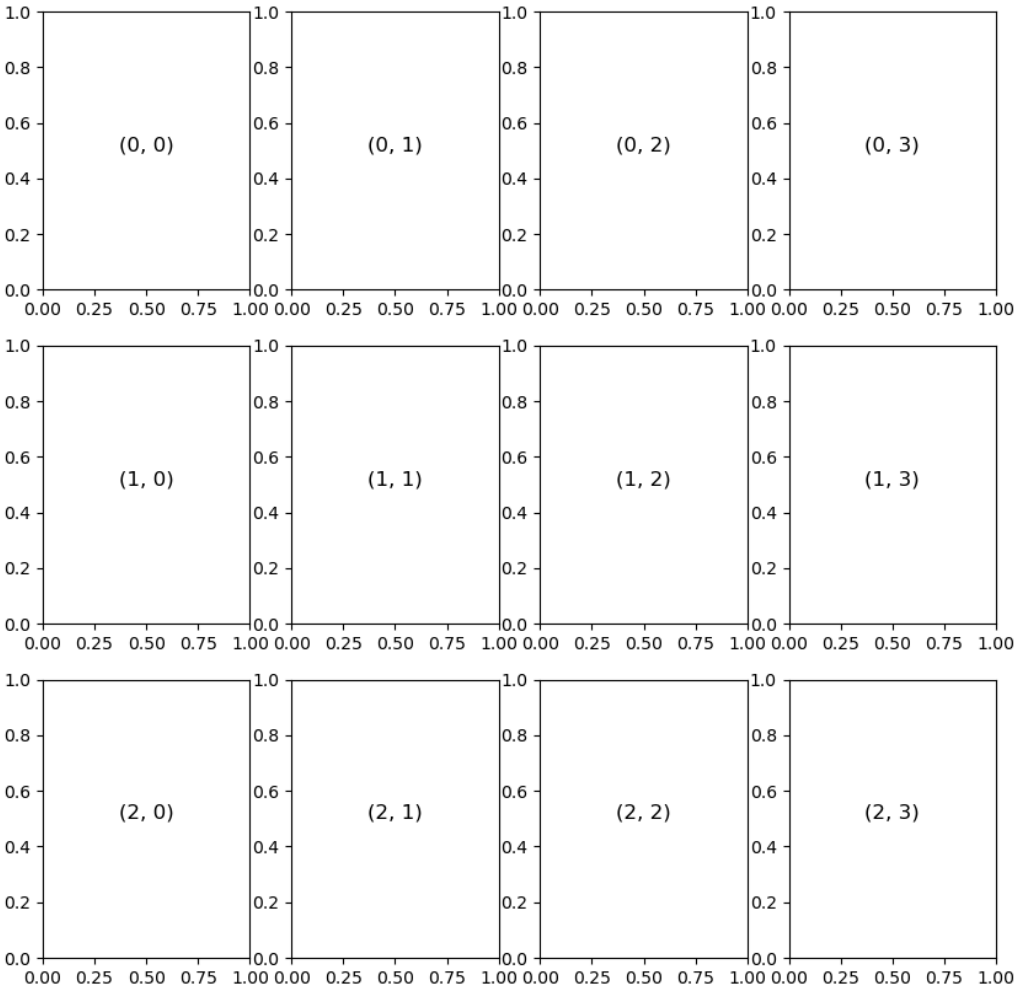
Change the nrows and ncols and see what you get!!!!

copy the script. Change nrows and ncols and see what happens

In [30]:
```
nrows=3  # CHANGE ME!
ncols=4  # CHANGE ME!
```

```
fig,ax=plt.subplots(nrows,ncols)
fig.set_size_inches(10,10)
for i in np.arange(nrows):
    for j in np.arange(ncols):
        ax[i, j].text(0.5, 0.5, str((i, j)),
                      fontsize=12, ha='center')
```

Figure



Follow me.

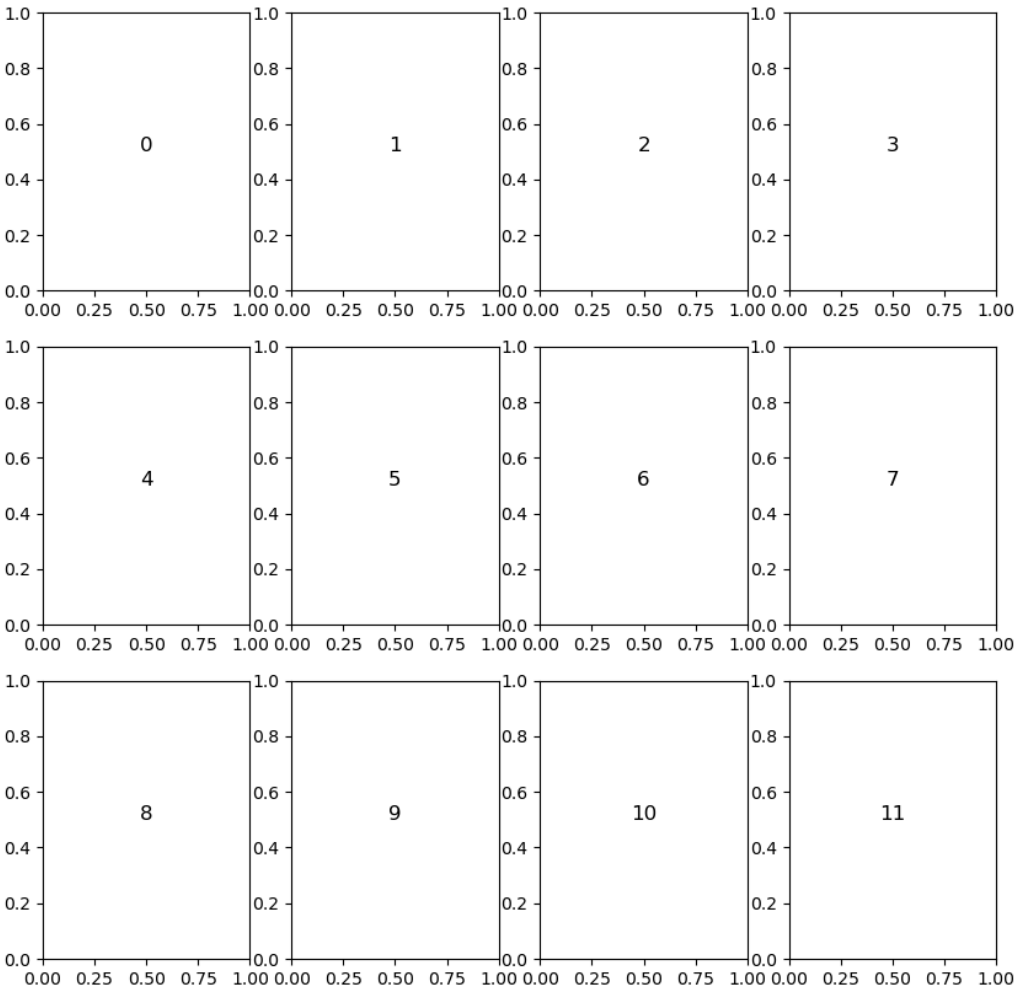
this is the hard part. We have a list of elements. we want plotted. It is a one-dimensional list with a length. But we want to put them on a two dimensional array of graphs. To continue our example above

- we are going to use np.ravel https://docs.scipy.org/doc/numpy/reference/generated/numpy.ravel.html
- this flattens an array. It takes a 2d array and makes it into a 1d array
- We can turn ax into a 1d array.

In [31]:
```
nrows=3
ncols=4
```

```
ngraphs=nrows*ncols
fig,ax2d=plt.subplots(nrows,ncols)
fig.set_size_inches(10,10)
ax=np.ravel(ax2d)

for count in np.arange(ngraphs):
    ax[count].text(0.5, 0.5, str((count)),
                       fontsize=12, ha='center')
```

Figure



Did you see that? Are you still with me?

Remember that we had our list of elements we wanted to plot?

Flashback

Do you remmeber I said we might want to count and give the value of an array?

Here is my quote from the notes.

Python has a simple trick to count through a list and give you the list. It may not make sense now but it will later in the semester! PAY ATTENTION TO THIS!

and I gave this example

In [32]:
```
mystrlist=['env','chem','bio','psych']
for count,mystr in enumerate(mystrlist):
    print (count,mystr)
```

```
0 env
1 chem
2 bio
3 psych
```

In [36]:
```

```

```
0 Sr
1 Ca
2 P
3 Fe
```

Lets automate it.

- make a new parameter called el_to_plot for elements to plot
- for now set to 4.
- now do an enumerate loop to print the top 4 elements
- then switch to 6

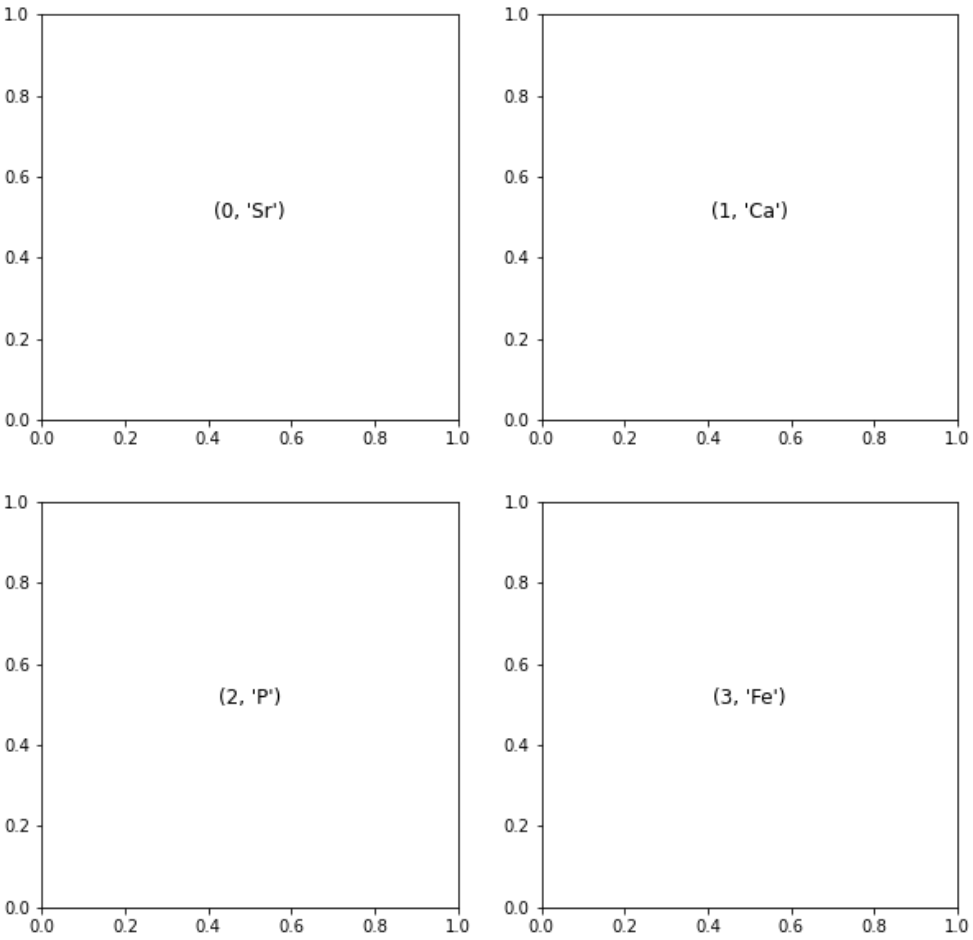
In [44]:
```

```

```
0 Sr
1 Ca
2 P
3 Fe
```

Can we get these on a graph.

- Grab the code from above to alter.
- Lets start by just printing the names.
- add the el_to_plot=4
- set your nrows. I chose 2.
-
- set your ncols=el_to_plot/nrows.
- If you get an error you need to make sure ncols in an int by ncols=int(el_to_plot/nrows).
- Use enumerate for your for loop like above going over the top r2 values.
- print the number and element on each graph
- remember you can just add things to print where it says str((count)) by adding in there. for example str((count,el))

In [50]:
```

```



Now go back up and add two more elements to your elems array and see if the graphs update.

Now you are ready!!!! Can you make these four plots???

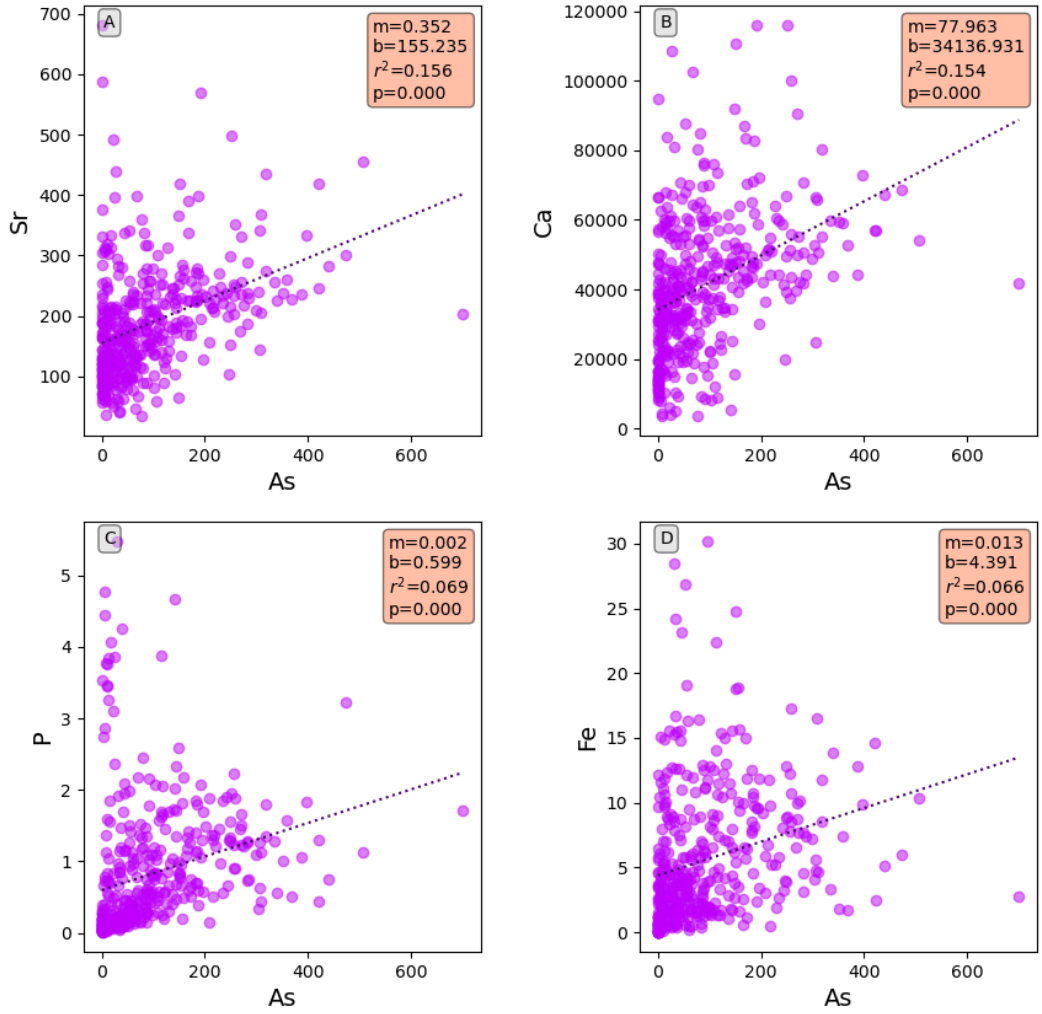
If your plots get too scrunched you can add this line after turning on the fig,ax to change whitespace and make it prettier.

A note on a function you might forget is how to get the spacing between graphs to look good. You might not remember that during the arsenic versus depth graphs I needed to add whitespace between the graphs to make them look good. I used this fucntion

fig.subplots_adjust(wspace=0.3)

In [51]:

Figure



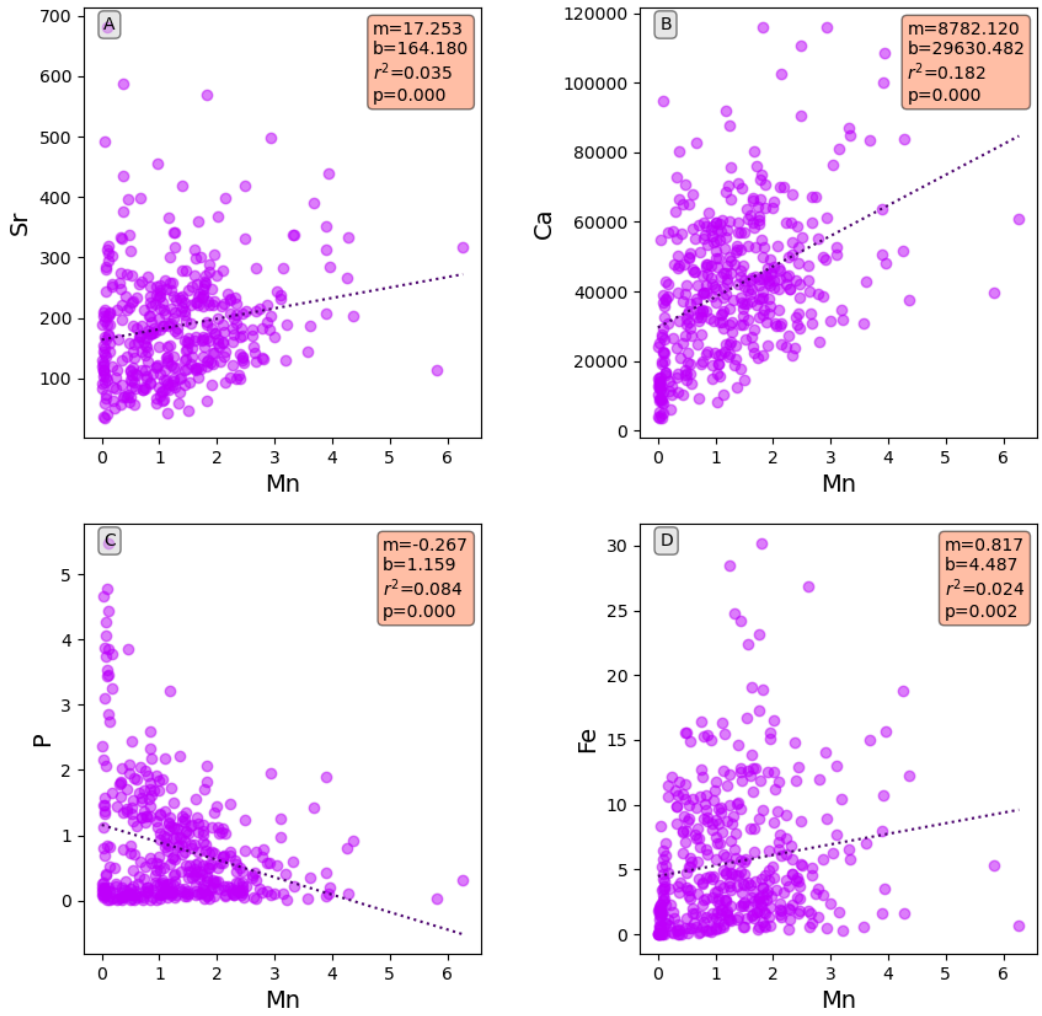
That is beautiful!

We are close to making this stand alone. i think we have removed most of the the hardwires.

- Make a filename str to use to save the figure. I added together the x and +'correlations.jpg'
- Save to a file fig.savefig(filename)
- Then change x to 'Mn' and see if it works.

In [53]:

Figure



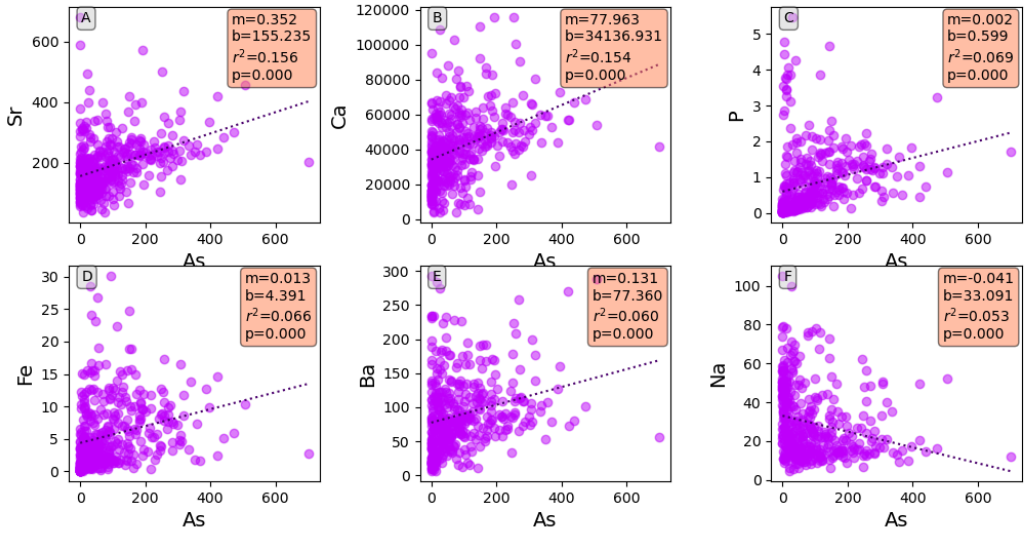
Can we do six plots?

- we should be able to just add two more elements to num_elements!

In [59]:
```
el_to_plot=6
nrows=2
ncols=int(el_to_plot/nrows)

fig,ax2d=plt.subplots(nrows,ncols)
fig.set_size_inches(12,6)
ax=np.ravel(ax2d)

fig.subplots_adjust(wspace=0.4)

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box
props2=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)  #for the letters

for count,element in enumerate(df_r2.index[0:el_to_plot]):

    x='As'
```

```
    y=element


    ax[count].scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
    ax[count].set_xlabel(x,fontsize=14)
    ax[count].set_ylabel(y,fontsize=14)

    df_clean=df[[x,y]].dropna()   #drop the not a numbers
    results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

    textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                    format(results.slope,results.intercept\
                          ,results.rvalue**2,results.pvalue)   #make the text string

    ax[count].text(0.97,0.97,textstr,transform=ax[count].transAxes,fontsize=10\
            ,verticalalignment='top',horizontalalignment='right'
            ,multialignment='left',bbox=props)     #add the text box

    x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
    ax[count].plot(x_fit,results.slope*x_fit+results.intercept
            ,color='xkcd:royal purple',linestyle='dotted')  #make the best fit line

    ax[count].text(0.05,0.98,chr(count + ord('A')),transform=ax[count].transAxes\
                   ,fontsize=10,verticalalignment='top',bbox=props2)  #plot the letters

filename=x+'_correlations.jpg' #Simpler than doing '{}_correlations.jpg'.format(x)
fig.savefig(filename)
```

Figure



Now you should be able to change number or rows and elements to plots and make a nice plot

This may not be a good thing. But you can now officially p-hack and make as many plots of your data as you would like!!!

For using seaborn I wanted a list of elements in order of just the values. You might use them later

In [19]:
```
df_r2.sort_values(by='r2',ascending=False,inplace=True)
```

In [57]:
```
df_r2.index.values
```

Out[57]:
```
array(['Sr', 'Ca', 'P', 'Fe', 'Ba', 'Na', 'Si', 'Mn', 'SO4', 'Br', 'S',
       'Cl', 'Lon', 'Mg', 'K', 'Lat', 'F'], dtype=object)
```

Also you can write your r2 values to excel. It is easy. just call .to_excel and give it a name with xlsx in it.

In [58]:
```
df_r2.to_excel('r2_class_example.xlsx')
```

In [ ]:
```

```

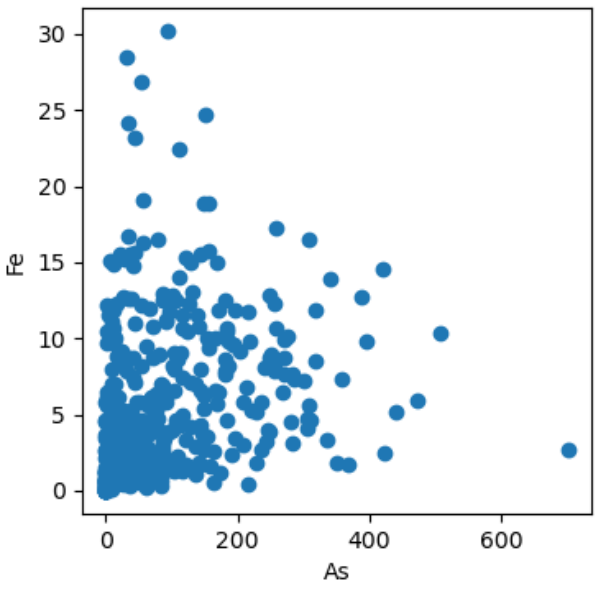
In [ ]:
```

```

ANSWERS

In [36]:
```
fig,ax=plt.subplots()
fig.set_size_inches(4,4)
x='As'
y='Fe'
print (y)
if df[y].dtype==float:
    ax.scatter(df[x],df[y])
    ax.set_xlabel(x)
    ax.set_ylabel(y)
```

```
Fe
```

Figure



The stats

In [37]:
```
x='As'
y='Fe'

results=stats.linregress(df[[x,y]].dropna())
textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                    format(results.slope,results.intercept\
                          ,results.rvalue**2,results.pvalue)
print (textstr)
```

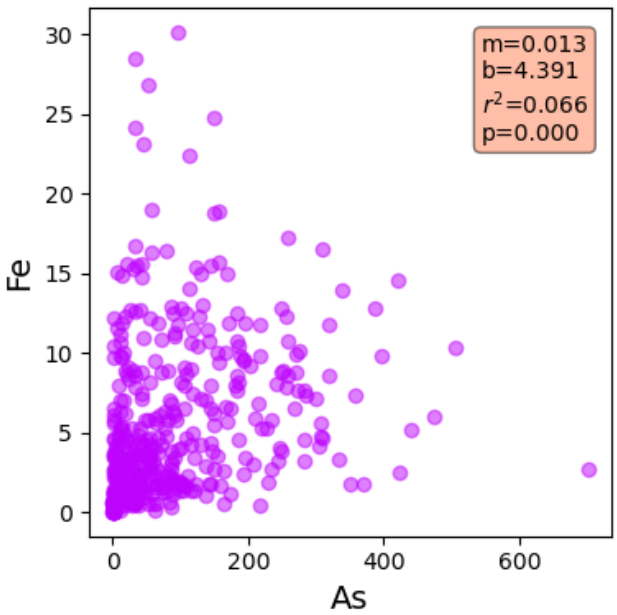
```
m=0.013
b=4.391
$r^2$=0.066
p=0.000
```

With the stats on the graph

In [34]:
```
fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

x='As'
y='Fe'

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box
print (y)

if df[y].dtype==float:
    ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
    ax.set_xlabel(x,fontsize=14)
    ax.set_ylabel(y,fontsize=14)

    df_clean=df[[x,y]].dropna()   #drop the not a numbers
    results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

    textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                    format(results.slope,results.intercept\
                          ,results.rvalue**2,results.pvalue)   #make the text string

    ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
            ,verticalalignment='top',bbox=props)     #add the text box
```

Fe

Figure



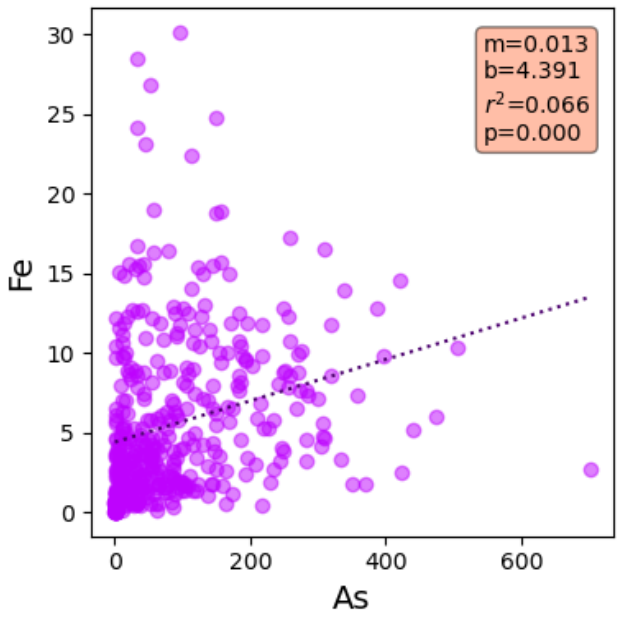
With the best fit line

In [39]:
```
fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

x='As'
y='Fe'
```

```
props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box
print (y)

if df[y].dtype==float:
    ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
    ax.set_xlabel(x,fontsize=14)
    ax.set_ylabel(y,fontsize=14)

    df_clean=df[[x,y]].dropna()   #drop the not a numbers
    results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

    textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                    format(results.slope,results.intercept\
                          ,results.rvalue**2,results.pvalue)   #make the text string

    ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
            ,verticalalignment='top',bbox=props)     #add the text box

    x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
    ax.plot(x_fit,results.slope*x_fit+results.intercept
            ,color='xkcd:royal purple',linestyle='dotted')  #make the plots
```

Fe

Figure



Making the pdf with the regression

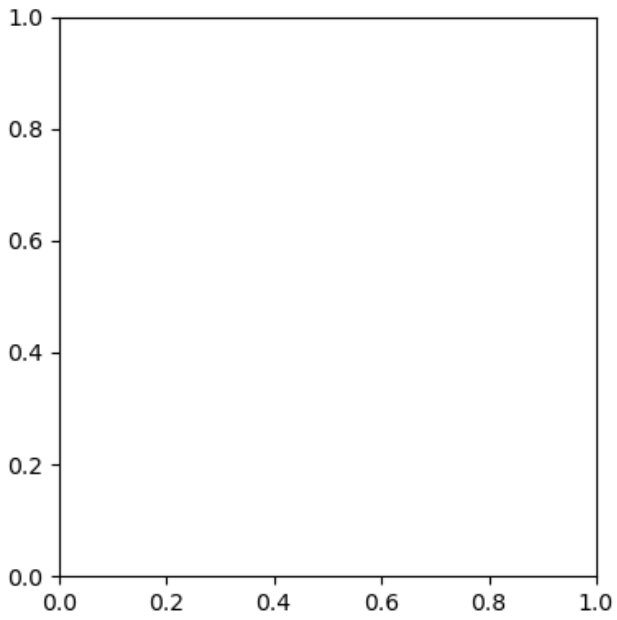
In [48]:

```
pp = PdfPages('regression.pdf')  #open a pdf file

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

for column in df:

    x='As'
    y=column

    print (y)

    if df[y].dtype==float and x!=y:
```

```
        ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
        ax.set_xlabel(x,fontsize=14)
        ax.set_ylabel(y,fontsize=14)

        df_clean=df[[x,y]].dropna()   #drop the not a numbers
        results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

        textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                        format(results.slope,results.intercept\
                              ,results.rvalue**2,results.pvalue)   #make the text string

        ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
                ,verticalalignment='top',bbox=props)     #add the text box

        x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
        ax.plot(x_fit,results.slope*x_fit+results.intercept
                ,color='xkcd:royal purple',linestyle='dotted')  #make the plots

        pp.savefig(dpi=200,bbox_inches='tight')
        ax.cla()  #clear the axes to start again

pp.close()  #close the pdf file when done
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



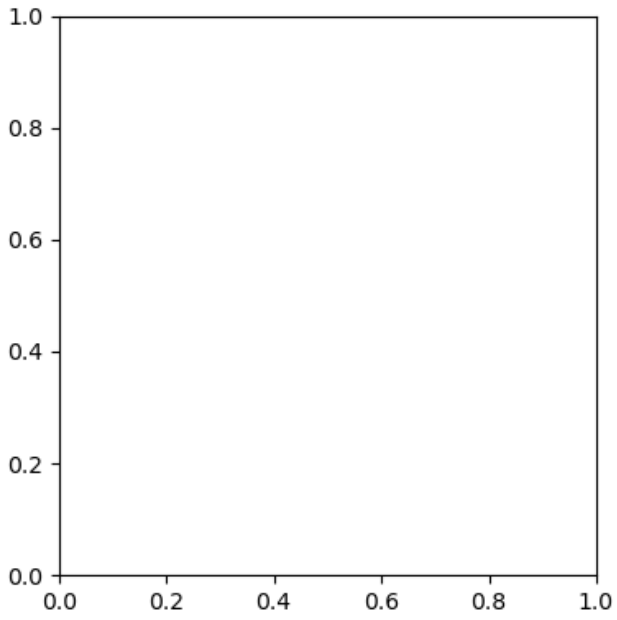
with a good title

In [53]:
```
pp = PdfPages('regression.pdf')  #open a pdf file

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

for column in df:

    x='As'
    y=column

    print (y)

    if df[y].dtype==float and x!=y:
        ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
        ax.set_xlabel(x,fontsize=14)
        ax.set_ylabel(y,fontsize=14)

        df_clean=df[[x,y]].dropna()   #drop the not a numbers
        results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

        textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                       format(results.slope,results.intercept\
                              ,results.rvalue**2,results.pvalue)   #make the text string

        ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
                ,verticalalignment='top',bbox=props)     #add the text box

        x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
        ax.plot(x_fit,results.slope*x_fit+results.intercept
                ,color='xkcd:royal purple',linestyle='dotted')  #make the plots

        if results.pvalue<0.01:
            title='{} vs {} is signifanct at the 0.01 level'.format(x,y)
            ax.set_title(title)
        else:
            title='{} vs {} is NOT signifanct at the 0.01 level'.format(x,y)
```

```
            ax.set_title(title)

        pp.savefig(dpi=200,bbox_inches='tight')
        ax.cla()  #clear the axes to start again

pp.close()  #close the pdf file when done
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



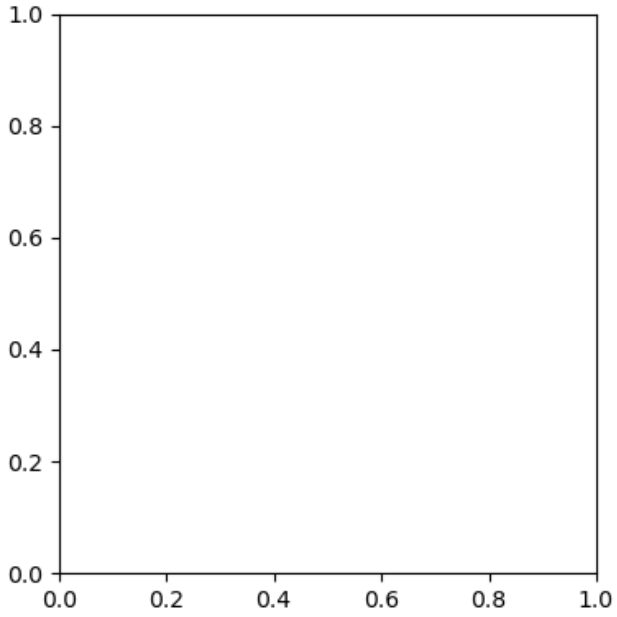
make a new dataframe and use the element as the index

In [54]:
```
pp = PdfPages('regression.pdf')  #open a pdf file

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box

fig,ax=plt.subplots(layout='tight')
fig.set_size_inches(4,4)

df_r2=pd.DataFrame() #empty dataframe

for column in df:
```

```
    x='As'
    y=column

    print (y)

    if df[y].dtype==float and x!=y:
        ax.scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
        ax.set_xlabel(x,fontsize=14)
        ax.set_ylabel(y,fontsize=14)

        df_clean=df[[x,y]].dropna()   #drop the not a numbers
        results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

        textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                       format(results.slope,results.intercept\
                              ,results.rvalue**2,results.pvalue)   #make the text string

        ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=10\
                ,verticalalignment='top',bbox=props)     #add the text box

        x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
        ax.plot(x_fit,results.slope*x_fit+results.intercept
                ,color='xkcd:royal purple',linestyle='dotted')  #make the plots

        if results.pvalue<0.01:
            title='{} vs {} is signifanct at the 0.01 level'.format(x,y)
            ax.set_title(title)
        else:
            title='{} vs {} is NOT signifanct at the 0.01 level'.format(x,y)
            ax.set_title(title)

        # Make a new dataframe with r2
        df_r2.at[column,'r2']=results.rvalue**2


        pp.savefig(dpi=200,bbox_inches='tight')
        ax.cla()  #clear the axes to start again

pp.close()  #close the pdf file when done
```

```
Well_ID
Lat
Lon
Depth
Drink
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



In [ ]:

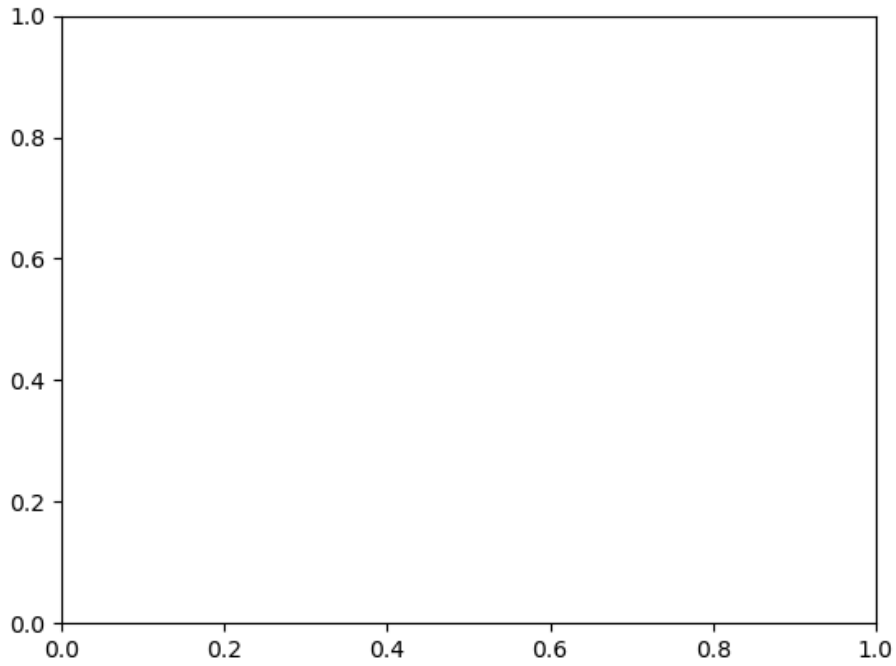
Answer for putting r^2 into a dataframe useing len

In [44]:
```
fig,ax=plt.subplots()
pp = PdfPages('As-corr.pdf')
df_r2=pd.DataFrame()

for col in df.iloc[:,5:]:

    x='As'
    y=col

    props=dict(boxstyle='round',facecolor='coral',alpha=0.5)
    print (y)

    if x!=y:
        ax.scatter(df[x],df[y])
        ax.set_xlabel(x,fontsize=14)
        ax.set_ylabel(y,fontsize=14)

        results=stats.linregress(df[[x,y]].dropna())

        textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                       format(results.slope,results.intercept\
                              ,results.rvalue**2,results.pvalue)

        index=len(df_r2)
        df_r2.at[index,'r2']=results.rvalue**2
        df_r2.at[index,'name']=y

        x_fit=np.array([df[x].min(),df[x].max()])
        ax.plot(x_fit,results.slope*x_fit+results.intercept,color='gray',linestyle='dotted')

        ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=14\
                ,verticalalignment='top',bbox=props)

        pp.savefig(bbox_inches='tight')
        ax.cla()
```

```
pp.close()
```

```
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



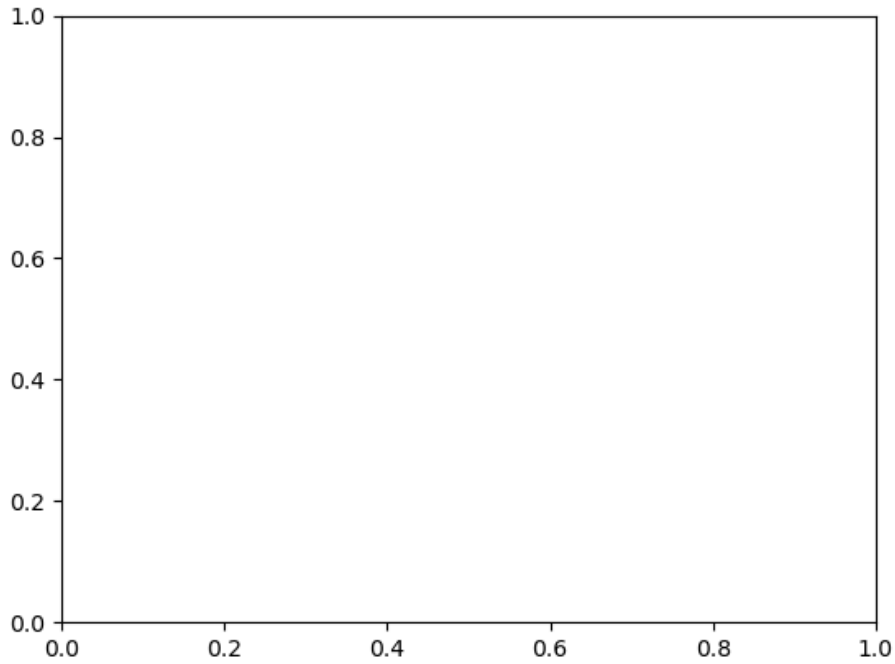
Answer for r^2 using enumerate

In [45]:
```
fig,ax=plt.subplots()
pp = PdfPages('As-corr.pdf')
df_r2=pd.DataFrame()

for count, col in enumerate(df.iloc[:,5:]):

    x='As'
    y=col

    props=dict(boxstyle='round',facecolor='coral',alpha=0.5)
    print (y)

    if x!=y:
        ax.scatter(df[x],df[y])
        ax.set_xlabel(x,fontsize=14)
```

```
        ax.set_ylabel(y,fontsize=14)

        results=stats.linregress(df[[x,y]].dropna())

        textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                        format(results.slope,results.intercept\
                              ,results.rvalue**2,results.pvalue)

        df_r2.at[count,'r2']=results.rvalue**2
        df_r2.at[count,'name']=y

        x_fit=np.array([df[x].min(),df[x].max()])
        ax.plot(x_fit,results.slope*x_fit+results.intercept,color='gray',linestyle='dotted')

        ax.text(0.75,0.95,textstr,transform=ax.transAxes,fontsize=14\
                ,verticalalignment='top',bbox=props)

        pp.savefig(bbox_inches='tight')
        ax.cla()

pp.close()
```

```
Si
P
S
Ca
Fe
Ba
Na
Mg
K
Mn
As
Sr
F
Cl
SO4
Br
```

Figure



to four elements

In [33]:
```
elements=['Sr','Ca','P','Fe']

for count,el in enumerate(elements):
    print(count,el)
```

```
0 Sr
1 Ca
2 P
3 Fe
```

automate top 4

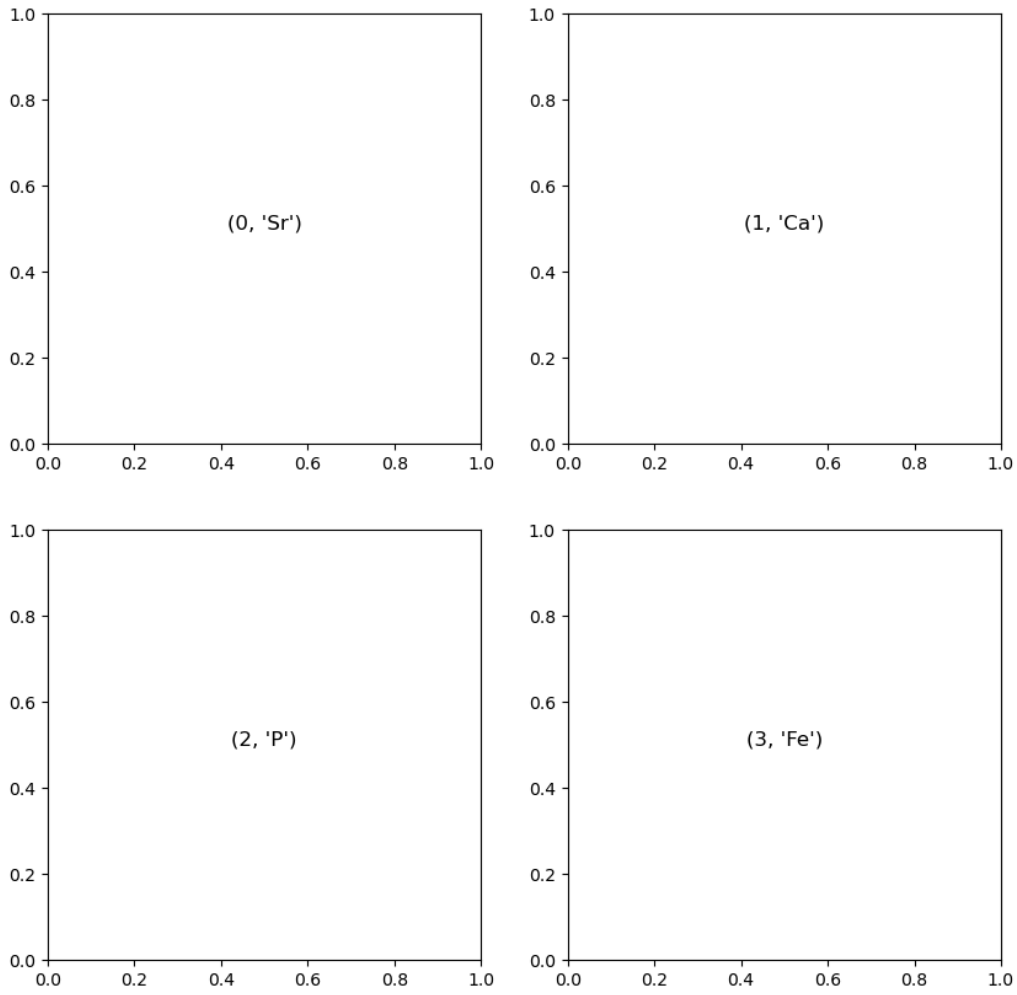
In [40]:
```
el_to_plot=4

for count,el in enumerate(df_r2.index[0:el_to_plot]):
    print(count,el)
```

```
0 Sr
1 Ca
2 P
3 Fe
```

Top four elements on a graph

In [42]:
```
el_to_plot=4
nrows=2
ncols=int(el_to_plot/nrows)

fig,ax2d=plt.subplots(nrows,ncols)
fig.set_size_inches(10,10)
ax=np.ravel(ax2d)

for count,element in enumerate(df_r2.index[0:el_to_plot]):
    ax[count].text(0.5, 0.5, str((count,element)),
                       fontsize=12, ha='center')
```

Figure

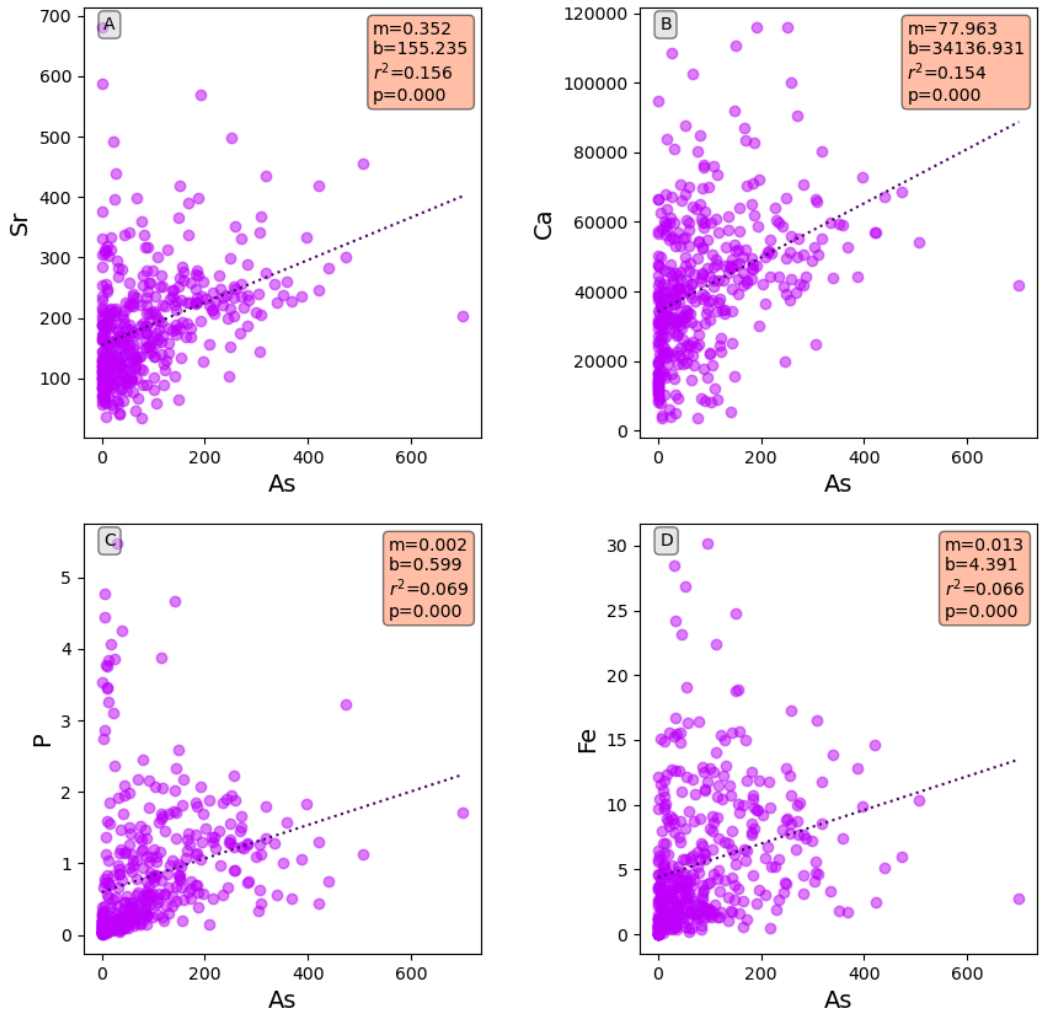


Top four graphs

In [50]:
```
el_to_plot=4
nrows=2
ncols=int(el_to_plot/nrows)

fig,ax2d=plt.subplots(nrows,ncols)
fig.set_size_inches(10,10)
ax=np.ravel(ax2d)

fig.subplots_adjust(wspace=0.4)

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box
props2=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)  #for the letters

for count,element in enumerate(df_r2.index[0:el_to_plot]):

    x='As'
    y=element
```

```
    ax[count].scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
    ax[count].set_xlabel(x,fontsize=14)
    ax[count].set_ylabel(y,fontsize=14)

    df_clean=df[[x,y]].dropna()   #drop the not a numbers
    results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

    textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                    format(results.slope,results.intercept\
                          ,results.rvalue**2,results.pvalue)   #make the text string

    ax[count].text(0.97,0.97,textstr,transform=ax[count].transAxes,fontsize=10\
            ,verticalalignment='top',horizontalalignment='right'
            ,multialignment='left',bbox=props)     #add the text box

    x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
    ax[count].plot(x_fit,results.slope*x_fit+results.intercept
            ,color='xkcd:royal purple',linestyle='dotted')  #make the best fit line

    ax[count].text(0.05,0.98,chr(count + ord('A')),transform=ax[count].transAxes\
                   ,fontsize=10,verticalalignment='top',bbox=props2)  #plot the letters
```

Figure



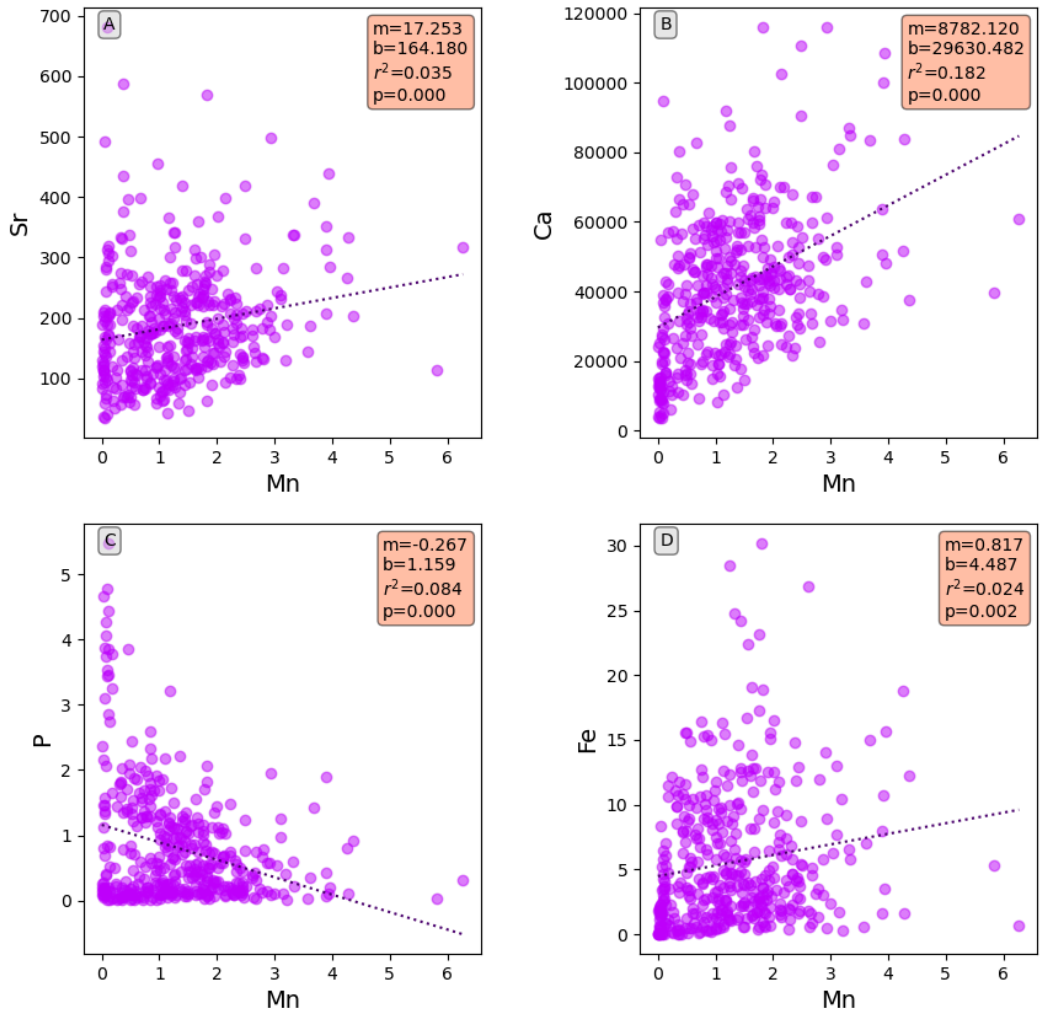
In [ ]:

Cleaned up and with Mn

In [52]:
```
el_to_plot=4
nrows=2
ncols=int(el_to_plot/nrows)

fig,ax2d=plt.subplots(nrows,ncols)
fig.set_size_inches(10,10)
ax=np.ravel(ax2d)

fig.subplots_adjust(wspace=0.4)

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box
props2=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)  #for the letters

for count,element in enumerate(df_r2.index[0:el_to_plot]):

    x='Mn'
```

```
    y=element


    ax[count].scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
    ax[count].set_xlabel(x,fontsize=14)
    ax[count].set_ylabel(y,fontsize=14)

    df_clean=df[[x,y]].dropna()   #drop the not a numbers
    results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

    textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                    format(results.slope,results.intercept\
                          ,results.rvalue**2,results.pvalue)   #make the text string

    ax[count].text(0.97,0.97,textstr,transform=ax[count].transAxes,fontsize=10\
            ,verticalalignment='top',horizontalalignment='right'
            ,multialignment='left',bbox=props)     #add the text box

    x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
    ax[count].plot(x_fit,results.slope*x_fit+results.intercept
            ,color='xkcd:royal purple',linestyle='dotted')  #make the best fit line

    ax[count].text(0.05,0.98,chr(count + ord('A')),transform=ax[count].transAxes\
                   ,fontsize=10,verticalalignment='top',bbox=props2)  #plot the letters

filename=x+'_correlations.jpg' #Simpler than doing '{}_correlations.jpg'.format(x)
fig.savefig(filename)
```

Figure

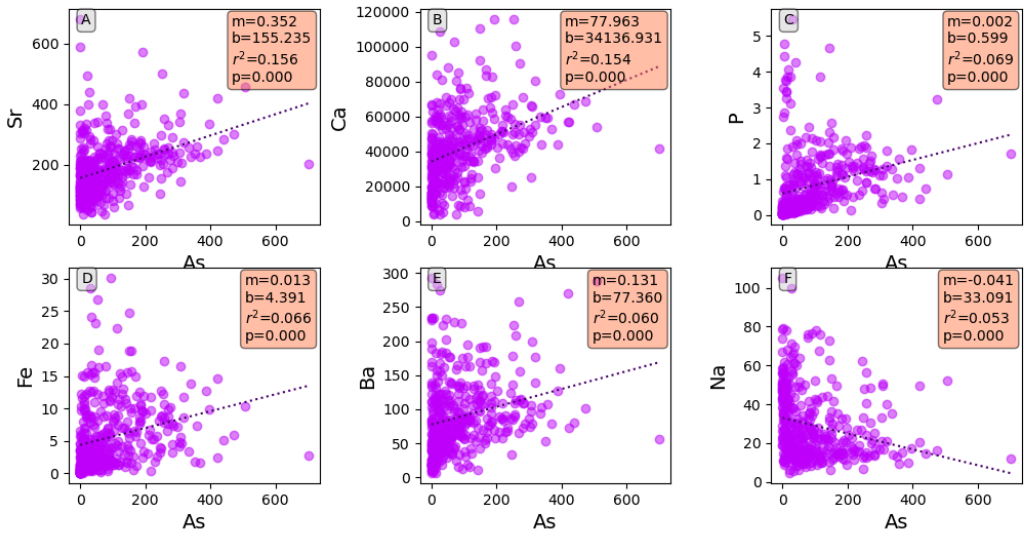


6 graphs with As

In [56]:
```
el_to_plot=6
nrows=2
ncols=int(el_to_plot/nrows)

fig,ax2d=plt.subplots(nrows,ncols)
fig.set_size_inches(12,6)
ax=np.ravel(ax2d)

fig.subplots_adjust(wspace=0.4)

props=dict(boxstyle='round',facecolor='coral',alpha=0.5)  #properties of the text box
props2=dict(boxstyle='round',facecolor='lightgrey',alpha=0.5)  #for the letters

for count,element in enumerate(df_r2.index[0:el_to_plot]):

    x='As'
    y=element
```

```
    ax[count].scatter(df[x],df[y],color='xkcd:bright purple',alpha=0.5)
    ax[count].set_xlabel(x,fontsize=14)
    ax[count].set_ylabel(y,fontsize=14)

    df_clean=df[[x,y]].dropna()   #drop the not a numbers
    results=stats.linregress(df_clean[x],df_clean[y])  #do the linregress

    textstr='m={:.3f}\nb={:.3f}\n$r^2$={:.3f}\np={:.3f}'.\
                    format(results.slope,results.intercept\
                          ,results.rvalue**2,results.pvalue)   #make the text string

    ax[count].text(0.97,0.97,textstr,transform=ax[count].transAxes,fontsize=10\
            ,verticalalignment='top',horizontalalignment='right'
            ,multialignment='left',bbox=props)     #add the text box

    x_fit=np.array([df_clean[x].min(),df_clean[x].max()])   #make the x values
    ax[count].plot(x_fit,results.slope*x_fit+results.intercept
            ,color='xkcd:royal purple',linestyle='dotted')  #make the best fit line

    ax[count].text(0.05,0.98,chr(count + ord('A')),transform=ax[count].transAxes\
                   ,fontsize=10,verticalalignment='top',bbox=props2)  #plot the letters

filename=x+'_correlations.jpg' #Simpler than doing '{}_correlations.jpg'.format(x)
fig.savefig(filename)
```

Figure



In [ ]: